User manual

`arabic-led-text-pipeline` — displaying correctly shaped Arabic on LED panels.

Covers wiring, first setup, daily use from a phone, configuration, desktop debugging, the wire protocol, and what to do when something is wrong.

Contents

1. What this is
2. Safety first — power
3. What you need
4. Wiring
5. Installing the toolchain
6. First flash
7. Bring-up: getting the picture the right way up
8. Everyday use from a phone
9. The controls, explained
10. Configuration reference
11. Desktop debugging
12. Wire protocol
13. Troubleshooting
14. Limits and known gaps

---

1. What this is

A system for showing Arabic text on inexpensive LED panels, driven from an ordinary phone with no app installed, no router, and no internet.

Arabic letters change shape depending on their neighbours, and ordinary LED sign controllers store one fixed picture per character — which is why so many Arabic signs show disconnected, unreadable letters. This system does the shaping on the phone, in the browser, and sends the panel nothing but finished pixels.

Supported panels:

Panel	Status
MAX7219 8×8 modules (single or grid)	Working, verified on hardware
WS2812B addressable RGB strips	Implemented, not yet verified on hardware
P10 DMD panels	Implemented from specification, **never tested**

---

2. Safety first — power

Read this before connecting anything, particularly for RGB strips.

WS2812B current draw

Each LED draws roughly **60 mA at full white** — 20 mA per colour channel.

Panel	At full white
8×8 (64 LEDs)	~3.8 A
16×16 (256 LEDs)	~15 A
32×8 (256 LEDs)	~15 A

A USB port supplies 0.5–2 A. **These numbers are far beyond it.**

The firmware protects against this: it estimates the current a frame would draw and scales the whole frame down proportionally if it exceeds the configured budget. The default budget is **2000 mA**, which is safe on a decent USB supply. When it clamps you will see:

```
W (12345) ws2812b: frame would draw ~4820mA, scaled to fit 2000mA budget
```

That is the protection working, not a fault. **Only raise `WS2812B_MAX_MA` to match a supply you actually own**, and remember the strip's own wiring has to carry that current too — thin jumper wires do not carry 10 A.

General

- **Always tie grounds together.** Voltage is a difference between two points; if the panel and the ESP32 do not share a ground reference, the data line means nothing to the receiver. The result is erratic behaviour rather than a clean failure, which makes it a confusing fault to chase.
- **Feed panels from their own 5 V supply** for anything sustained, not from the ESP32's regulator.
- **Never use 3.3 V for panel VCC.** LEDs are dim and logic thresholds stop matching, giving flicker and garbage instead of an honest failure.

---

3. What you need

- An **ESP32** development board (classic ESP32; this was built on an ESP32-D0WD-V3)
- One of the supported panels
- Five jumper wires (three for a WS2812B strip)
- A **data** USB cable — charge-only cables are a classic time sink
- A computer running Linux, macOS or Windows

---

4. Wiring

MAX7219

All five wires land on **one edge of the board** — the edge carrying VIN and GND.

MAX7219	ESP32	Notes
VCC	**VIN (5 V)**	Not 3V3
GND	**GND**	
DIN	**GPIO13**	HSPI native MOSI
CLK	**GPIO14**	HSPI native SCK
CS	**GPIO27**	See the warning below

WS2812B

Strip	ESP32	Notes
5V / VCC	**5 V supply**	External supply for anything beyond a few LEDs
GND	**GND**	Must be shared with the ESP32
DIN	**GPIO27**	Data flows DIN → DOUT along the strip

Connect at the **DIN** end. The DOUT at the far end goes to the next strip section, or nowhere.

P10 (HUB12) — untested

HUB12	ESP32
R (data)	GPIO13
CLK	GPIO14
LAT / STB	GPIO27
A	GPIO26
B	GPIO25
OE	GPIO33
GND	GND

P10 panels need their own 5 V supply; do not power them from the ESP32.

⚠ Do not use GPIO12

On most DevKits GPIO12 sits invitingly between GPIO13 and GPIO14. **Avoid it.**

GPIO12 is a *strapping pin*: the ESP32 reads it at power-up to choose its internal flash voltage, and it must read LOW at that instant. A wire holding it high at boot makes the chip select the wrong flash voltage and **refuse to start**. The board appears dead, and nothing about the symptom points at your wiring.

Finding the pins

Read the labels silkscreened on your board rather than counting positions. ESP32 DevKits ship in 30-pin and 38-pin variants whose physical ordering differs, but all of them print the pin names.

---

5. Installing the toolchain

```
# Dependencies (Debian/Ubuntu)
sudo apt install git wget flex bison gperf python3 python3-pip python3-venv \
                 cmake ninja-build ccache libffi-dev libssl-dev dfu-util libusb-1.0-0

# The SDK — --recursive matters, it pulls in FreeRTOS, lwIP and mbedTLS
mkdir -p ~/esp && cd ~/esp
git clone -b v5.3.2 --recursive https://github.com/espressif/esp-idf.git
cd esp-idf && ./install.sh esp32

# Activate — needed in EVERY new terminal
. ./export.sh
```

Without `--recursive` you get an incomplete SDK and a confusing build failure. Forgetting `export.sh` gives `idf.py: command not found` — that is all it ever means.

Serial port permission (Linux)

```
ls -l /dev/ttyUSB0              # is the board visible?
groups | grep dialout           # are you permitted to use it?
sudo usermod -aG dialout $USER  # if not — then log out and back in
```

---

6. First flash

```
cd firmware
idf.py set-target esp32
idf.py menuconfig          # → Arabic LED text pipeline → Display type
idf.py build
idf.py -p /dev/ttyUSB0 flash monitor
```

You never press a button on the board. The USB-serial chip's control lines reset the ESP32 and put it into its bootloader automatically.

A healthy start looks like:

```
I (516) max7219: ready: 1x1 module(s) = 8x8 px, clk=14 din=13 cs=27
I (521) main: display: MAX7219, 8x8, colour=no
I (727) wifi_ap: AP "ARABIC-LED" up (open), http://192.168.4.1/
I (742) wifi_ap: captive DNS listening on :53
I (748) http_ui: web UI on http://192.168.4.1/ (panel 8x8, colour=no)
I (755) main: self-test: scrolling "HELLO WORLD 0123456789" (131px) across 8x8
```

The Latin self-test exists to prove the whole chain — wiring, transport, geometry, packing — using text whose correct appearance needs no interpretation. **If English does not render correctly, Arabic never will**, so fix that first.

---

7. Bring-up: getting the picture the right way up

MAX7219 only. Skip this for WS2812B, where the layout is a setting rather than a discovery.

Generic MAX7219 modules wire their driver chip to the LED grid differently, and **it cannot be detected in software** — the chip has no readback line. It has to be observed.

The corner probe

Leave `MAX7219_ORIENTATION` at **-1** and the firmware boots into a probe that lights **one corner at a time**, three seconds each, logging which corner it intends:

```
corner 1/4: logical TOP-LEFT      (x=0 y=0)
corner 2/4: logical TOP-RIGHT     (x=7 y=0)
corner 3/4: logical BOTTOM-RIGHT  (x=7 y=7)
corner 4/4: logical BOTTOM-LEFT   (x=0 y=7)
--- cycle end, blanking for 2s ---
```

Write down where each dot actually appears on your panel. Four observations determine the answer exactly — there is no guessing.

Worked example

On the panel this was developed against:

Logical corner	Actually appeared at
top-left	bottom-left
top-right	top-left
bottom-right	top-right
bottom-left	bottom-right

That is a 90° rotation — the module is column-addressed with bit 0 at the top, which is `transpose + flipY` = **orientation 5**.

Setting it

```
idf.py menuconfig    # → Module pixel orientation → 5
idf.py -p /dev/ttyUSB0 flash
```

The eight values are the symmetries of a square:

0 plain	1 transpose	2 flipX	3 transpose+flipX
4 flipY	5 transpose+flipY	6 flipX+flipY	7 transpose+flipX+flipY

If none looks right, check the **chain layout** setting instead — for a grid of modules, a serpentine ribbon wired as row-major scrambles alternate rows.

---

8. Everyday use from a phone

1. **Power the board.** It advertises an open WiFi network named `ARABIC-LED`.
2. **Join that network** from the phone. No password by default.
3. A **sign-in notification** should appear — tap it. If it does not, open `http://192.168.4.1/` in any browser.
4. **Type your text**, watch the preview, press **إرسال إلى اللوحة** (send to panel).

No app, no router, no internet, no SIM. The board serves the page from its own flash memory.

> **If the notification does not appear**, the page still works by typing the address. Modern Android and iOS increasingly check connectivity over HTTPS, which cannot be intercepted. See Troubleshooting.

---

9. The controls, explained

النص — Text

Type Arabic, Latin, or both. **The shaping happens in your phone's browser**, using the same text engine the rest of your phone uses, so contextual letter forms and the `لا` / `الله` ligatures come out correct.

المعاينة — Preview

Shows exactly what the panel will display, pixel for pixel. In scrolling mode it animates at the real speed, so you can judge the speed setting directly rather than imagining it.

العرض — Display mode

Mode	Behaviour
ثابت (static)	The message is fitted to the panel and held
متحرك (scrolling)	The whole phrase is sent at full width and scrolls across

If you choose scrolling but the text already fits, it says so rather than silently doing nothing.

اتجاه الحركة — Scroll direction

This matters for Arabic and is not cosmetic.

- **يمين (Arabic)** — text travels rightward. Arabic reads right-to-left, so its first character sits at the *right* end of the visual string and must travel right to be revealed in reading order.
- **يسار (Latin)** — text travels leftward, the familiar marquee direction.

It is a manual setting because a mixed Arabic/Latin phrase has no single correct answer, and you are better placed to decide than a rule of thumb.

سرعة الحركة — Speed

Milliseconds per one-pixel step. Lower is faster. 60 ms is a comfortable reading pace.

حجم الخط — Font size

Pixel height of the text. On an 8-pixel panel, 8 fills it completely.

سماكة الخط — Stroke weight

This is the one that needs explaining.

Your browser draws text **anti-aliased** — the edges come out grey. A monochrome panel has no grey, only lit or unlit. This slider sets the cutoff where a grey pixel becomes lit.

- **Lower value → fatter strokes** (more greys count as lit)
- **Higher value → thinner strokes**

At 8 pixels tall this is the difference between a letter that reads and a blob. The right value differs by font and by size, which is exactly why it is a control and not a fixed constant.

اللون — Colour *(colour panels only)*

Appears only when the attached panel can show colour. The page asks the board what is connected and hides the control otherwise.

On a colour panel the pipeline **skips thresholding**: coverage becomes brightness against your chosen colour, so anti-aliased edges survive as dimmer pixels rather than being forced on or off. Letterforms keep their shape better as a result — the real advantage of colour at this size, more than the colour itself.

الخط — Font

Which typeface the browser uses. Availability differs by device: what your phone has and what your laptop has are not the same set, so the same text can render slightly differently on each.

The diagnostic line

```
ذروة 214 · معامل 1.19 · 37 نقطة مضاءة
```

Peak brightness found, the gain applied to normalise it, and how many pixels ended up lit. Useful when comparing two devices: if these numbers differ a lot between your phone and laptop, that is the font differing, not a fault.

---

10. Configuration reference

All under `idf.py menuconfig` → **Arabic LED text pipeline**.

Display type

Which panel is attached: MAX7219, WS2812B, or P10. Everything above the driver is identical for all three — only geometry, colour and current limiting differ.

MAX7219 wiring

Setting	Default	Meaning
`MAX7219_PIN_CLK`	14	Clock
`MAX7219_PIN_DIN`	13	Data
`MAX7219_PIN_CS`	27	Chip select
`MAX7219_COLS`	1	Modules across; panel is `COLS × 8` wide
`MAX7219_ROWS`	1	Modules down; panel is `ROWS × 8` tall
`MAX7219_CHAIN`	row-major	How the ribbon runs between rows
`MAX7219_INTENSITY`	2	0–15
`MAX7219_ORIENTATION`	-1	-1 runs the corner probe; else 0–7

WS2812B strip

Setting	Default	Meaning
`WS2812B_PIN`	27	Data pin
`WS2812B_COLS`	8	LEDs across
`WS2812B_ROWS`	8	Rows; 1 for a straight strip
`WS2812B_SERPENTINE`	on	Strip folds at each row end, so alternate rows run backwards
`WS2812B_MAX_MA`	2000	Current budget — **see Safety**

Serpentine vs progressive: serpentine means the strip snakes back and forth and needs no return wires — how most hand-built matrices are made. Progressive means every row runs the same direction with a wire returning from each row's end to the next row's start. Choosing wrong scrambles alternate rows and looks like a corrupt font.

P10 panel *(untested)*

Pins `P10_PIN_R/CLK/LAT/A/B/OE`, panel counts `P10_PANELS_X/Y` (each panel is 32×16), `P10_BRIGHTNESS` 0–255, and `P10_INVERT` — many P10 panels light on a **zero** bit, so if the image comes out as a photographic negative, flip that.

WiFi access point

Setting	Default
`AP_SSID`	`ARABIC-LED`
`AP_PASSWORD`	empty (open network)

A password shorter than 8 characters is *invalid* rather than weak — WPA2 rejects it — so the firmware falls back to an open network rather than failing to start.

Self-test

`SELFTEST_ENABLE`, `SELFTEST_TEXT` (default `HELLO WORLD 0123456789`), `SELFTEST_SCROLL_MS`, and `SELFTEST_WORD` (the single glyph shown during the corner probe).

Frame link

`FRAME_UART_ENABLE` (off by default), `FRAME_UART_NUM`, `FRAME_UART_BAUD`, and `FRAME_MAX_PAYLOAD` (12288) which bounds the longest message. RGB frames are 24× the size of monochrome ones.

> Putting the UART frame receiver on **UART0** conflicts with the console that logs are written through, and corrupts log output. Use UART1 or UART2 with their own pins if you enable it.

---

11. Desktop debugging

Testing from a phone means no developer tools and an awkward loop. The bridge serves the firmware's **own** page on your computer and forwards frames over the USB cable:

```
python3 host/debug_bridge.py --port /dev/ttyUSB0 --watch
# then open http://127.0.0.1:8080/
```

Option	Purpose
`--port`	Serial device
`--width` / `--height`	Panel size to report to the page
`--colour`	Pretend the panel shows colour, to exercise the RGB path
`--watch`	Stream the device's log into the same terminal

The page served is byte-for-byte the one that ships — only the transport differs. Edit `firmware/main/www/index.html`, reload, iterate.

It also prints each frame as ASCII art:

```
  → 8x8px, 192B payload, rgb, static, crc=8c09f487
    |++++++++|
    |+       |
    |+++++   |
```

If that art and the physical panel disagree, the fault is the driver's geometry, not the browser. That is the whole diagnostic value.

Requires `FRAME_UART_ENABLE` in menuconfig. Note the bridge holds the serial port, so stop it before flashing.

---

12. Wire protocol

Documented so that **any** microcontroller can implement a receiver without reading this project's source. That is a design goal, not a courtesy.

Frame layout

```
SOH | ver | w | h | fmt | flags | speed | R | G | B | payload | crc32
0x01   2                                                 4 bytes, LE
```

Field	Size	Meaning
`SOH`	1	Start of frame, `0x01`
`ver`	1	Format version, currently **2**. Reject anything else
`w`	1	Width — in **bytes** for mono, in **pixels** for RGB
`h`	1	Height in pixels
`fmt`	1	`0` = 1 bpp monochrome, `1` = 24 bpp RGB
`flags`	1	bit 0 = scroll, bit 1 = scroll rightward
`speed`	1	Milliseconds per 1-pixel scroll step
`R G B`	3	Colour of a lit pixel — **monochrome frames only**
`payload`	varies	`w × h` for mono; `w × h × 3` for RGB
`crc32`	4	Little-endian, over the 9 header bytes **and** the payload

Pixel packing (monochrome)

One bit per pixel, row-major, **most significant bit first**: bit 7 of byte 0 is the top-left pixel.

```
bool lit = payload[y * w + (x / 8)] & (0x80 >> (x % 8));
```

CRC

CRC-32/ISO-HDLC — polynomial `0xEDB88320` reflected, initial value `0xFFFFFFFF`, final XOR `0xFFFFFFFF`. This is exactly what Python's `zlib.crc32` produces, which makes host tooling a one-liner rather than a reimplementation.

```
crc = zlib.crc32(header + payload) & 0xFFFFFFFF
```

Behaviour a receiver should implement

- **Wrong version → reject.** Misparsing produces plausible garbage on the panel; rejecting produces a log line naming the problem.
- **Bad CRC → drop the frame and keep displaying the previous one.** On a notice board a briefly stale message beats a blanking or flickering panel.
- **Scrolling frames are wider than the panel.** The whole phrase arrives once and the receiver windows across it, so animation costs no further traffic.

HTTP alternative

```
POST /frame?w=<n>&h=<n>&fmt=<0|1>&rgb=<hex>&mode=<static|scroll>&dir=<rtl|ltr>&speed=<ms>
```

with the raw payload as the body. No CRC — TCP already checksums.

`GET /panel` returns `{"width":8,"height":8,"colour":false}` so a client can adapt.

---

13. Troubleshooting

Nothing lights at all

1. **Power** — 5 V not 3.3 V, and is the supply adequate?
2. **Grounds tied together?** This is the most common cause.
3. **Is the board even booting?** Run `idf.py monitor`. If there is no output, it is not a display problem.
4. **CS floating** (MAX7219) — check that wire.

The board will not boot with the panel connected

Almost certainly **GPIO12**. Move that wire. See Wiring.

Text is mirrored, rotated or upside-down

MAX7219 orientation. Set it to -1, run the corner probe, and derive the answer from the four observations. See Bring-up.

Every other row is scrambled

Chain layout. A grid wired serpentine but configured row-major (or the reverse) reverses alternate rows. Flip the setting.

The image is a photographic negative

P10 only — flip `P10_INVERT`.

The panel is dimmer than expected

If it is a WS2812B, check the log for the current clamp:

```
W (12345) ws2812b: frame would draw ~4820mA, scaled to fit 2000mA budget
```

That is protection, not a fault. Either accept the dimming, use fewer lit pixels or a darker colour, or supply more current *and* raise `WS2812B_MAX_MA` to match.

Pixels are missing on one device but not another

Almost always **font differences**. Compare the diagnostic line (`ذروة … معامل …`) between the two devices — if peak and gain differ noticeably, the two are rasterising different outlines. Adjust **سماكة الخط** for that device.

The sign-in notification never appears

The page still works at `http://192.168.4.1/`. Portal detection relies on intercepting the phone's connectivity check, and modern Android and iOS increasingly perform that check over **HTTPS**, which cannot be intercepted without a certificate the phone would reject.

The clean fix is RFC 8910 DHCP option 114, which *announces* the portal rather than tricking the phone into noticing one — that needs **ESP-IDF 5.4 or newer**.

Arabic letters do not join

If the *preview on your phone* shows them joined but the panel does not, it is a rendering or threshold problem — adjust **سماكة الخط** or increase the font size.

If the **preview itself** shows them disconnected, your browser is not shaping the text. Try a different browser; this is the one part of the pipeline this project does not implement itself.

`idf.py: command not found`

Run `. ~/esp/esp-idf/export.sh` — it applies per terminal.

`Could not open /dev/ttyUSB0`

The port is busy or the board is unplugged. Check `ls /dev/ttyUSB*`, and stop the debug bridge or a serial monitor if one is running.

---

14. Limits and known gaps

Stated plainly.

- **P10 has never been tested.** Written against the HUB12 specification; the byte ordering these panels use varies by manufacturer. Treat it as unproven until it has been through the corner-probe bring-up.
- **WS2812B is unverified on hardware** at the time of writing. The code builds and the geometry is proven in simulation, but no strip has been driven.
- **Output is device-dependent.** Deterministic downsampling removed most cross-device variation, but phones and laptops do not ship the same fonts, so identical text can produce different outlines. The pixel-exact preview guarantee holds **per device**; it does not hold *across* devices.
- **Arabic legibility below about 8 pixels is genuinely hard**, and no published work was found on Arabic glyph design at these sizes. Whether readable Arabic at this scale needs purpose-designed letterforms is an open question.
- **The captive portal is best-effort** where the phone probes over HTTPS.
- **The wire format is provisional.** Version 2 is current, but the framing is not yet frozen — resynchronisation after a desync relies on the CRC rejecting false starts.

Getting help

- Architecture and how the parts work: `foundations.md`
- Prior work and where this sits: `related-work.md`
- Wiring diagram: `wiring.md`
- Bring-up detail: `bringup.md`

Licensed **AGPL-3.0-or-later**. If you use it in research, please cite it — see `CITATION.cff`.